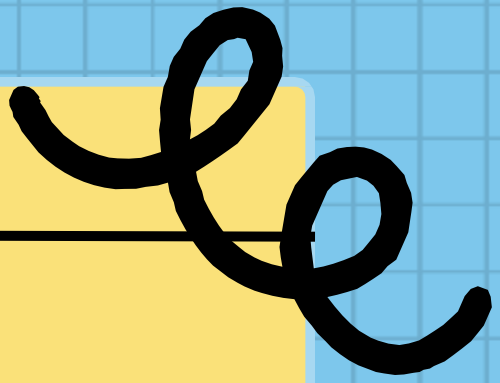
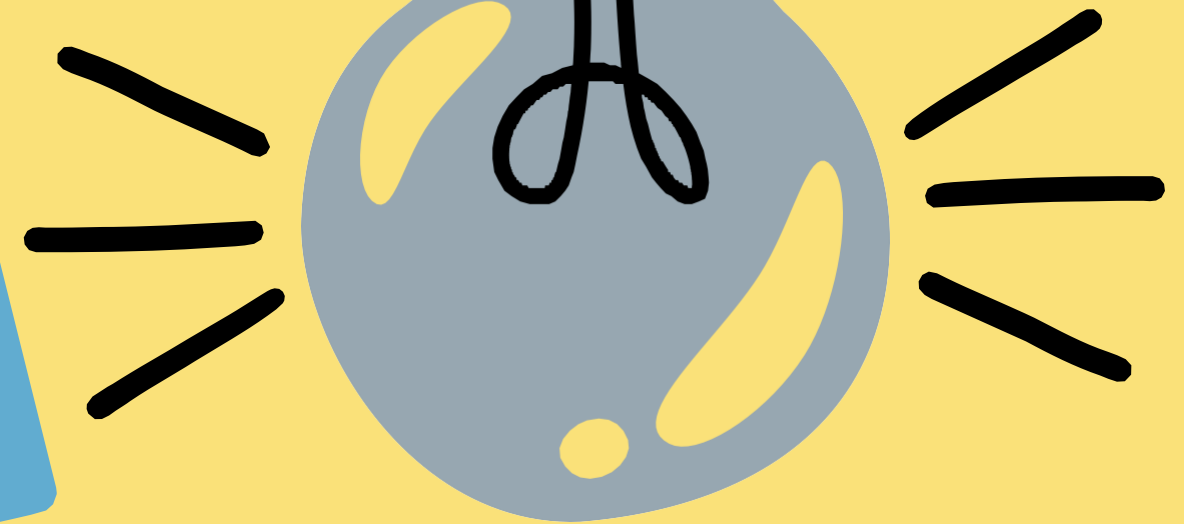


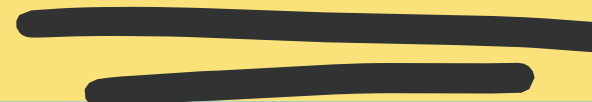
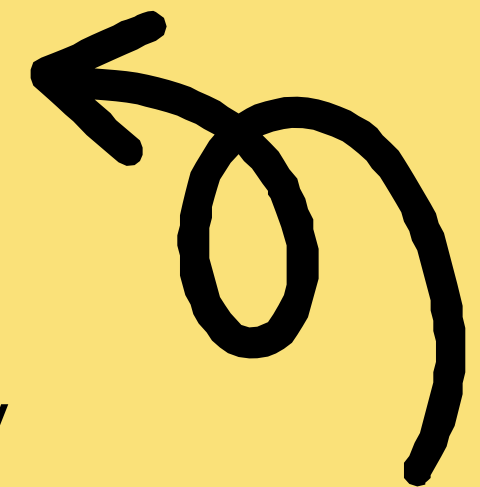
ESQUEMA de TRADUCCIÓN



¿QUÉ SON?



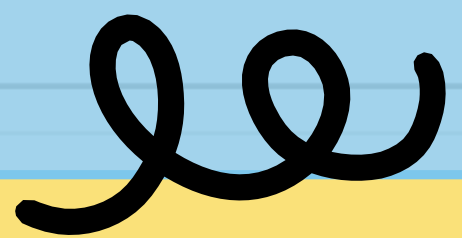
Los esquemas de traducción son una técnica en la que se asocian atributos con símbolos gramaticales y se insertan acciones semánticas (fragmentos de código entre llaves `{}`) en las reglas de producción de una gramática independiente del contexto. Estas acciones semánticas actúan sobre y modifican los atributos asociados a los nodos del árbol sintáctico, el orden en que se ejecutan estas acciones sigue un recorrido del árbol de análisis sintáctico.



PASOS A SEGUIR



Para aplicar un esquema de traducción, primero se debe construir el árbol sintáctico del programa y luego ejecutar las acciones semánticas incrustadas en las reglas. Los esquemas de traducción son una notación complementaria a las definiciones dirigidas por la sintaxis. Estos esquemas utilizan una gramática incontextual con fragmentos de código (llamados acciones semánticas) incrustados en las producciones gramaticales.

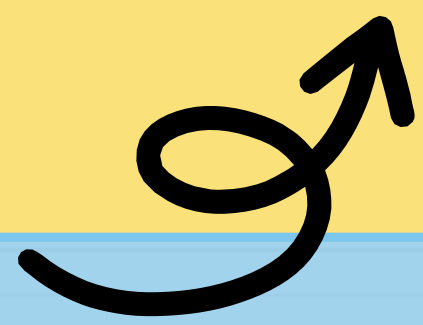


...

CONSTA DE:

...

Reglas gramaticales: Estas reglas definen la estructura sintáctica del lenguaje de programación. Pueden ser reglas de producción en una gramática libre de contexto.

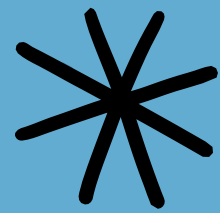




CONSTA DE:

Acciones semánticas: Estas acciones describen qué hacer cuando se aplica una regla gramatical específica. Pueden ser expresiones en un lenguaje de programación (como C, Java o un lenguaje específico del compilador) que calculan o asignan valores a los atributos semánticos.

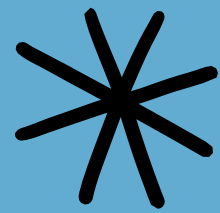
ACCIONES SEMÁNTICAS



Se utilizan para
realizar
operaciones
como:

- Asignación de valores a atributos:
Por ejemplo, puedes asignar un valor a un atributo de un nodo del árbol sintáctico, como en **`$expr{tac} = new_temp();`**, donde se está asignando un nuevo temporizador (en notación de tres direcciones) al atributo **`$expr{tac}`**.
- Generación de código intermedio:
Puedes generar código intermedio en notación de tres direcciones (TAC) o en otro formato, como en **`$term{tac} = new_temp(); gen($term{tac}, "=", $term[1]{tac}, "*", $factor{tac});`**, donde se está generando código TAC para una expresión aritmética.

ACCIONES SEMÁNTICAS



Se utilizan para
realizar
operaciones
como:

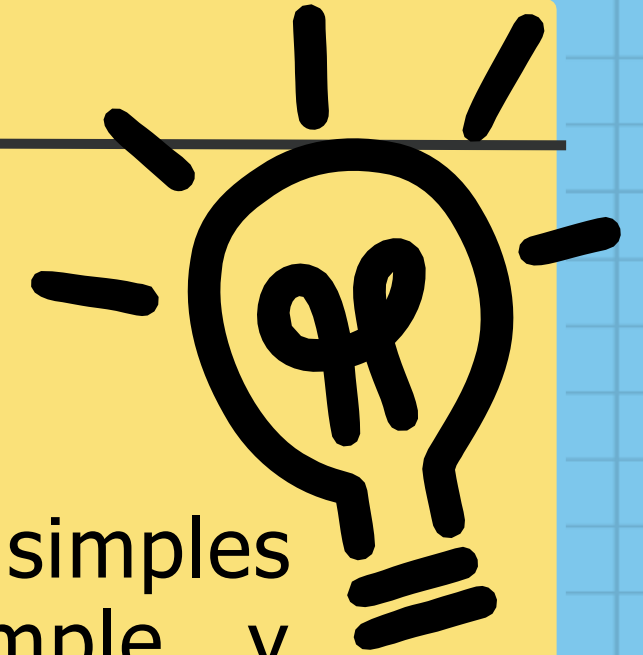
- Creación de estructuras de datos:

Puedes crear y manipular estructuras de datos utilizadas para representar el árbol de análisis sintáctico o realizar otros cálculos relacionados con la semántica del programa.

- Validaciones semánticas:

Puedes realizar comprobaciones y validaciones semánticas, como la verificación de tipos o la detección de errores semánticos en el código fuente.

EJEMPLO 1



Supongamos que queremos traducir expresiones aritméticas simples del español al inglés. Utilizaremos una gramática simple y asumiremos que la operación principal es la suma. La gramática podría ser algo así:

Expresión en español: "2 + 3"

Gramática:

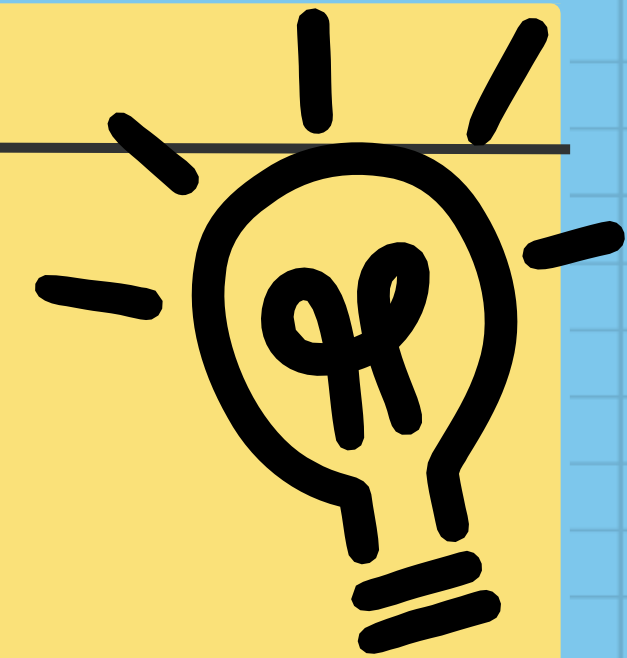
Exp $\rightarrow$ Exp '+' Term

Exp $\rightarrow$ Term

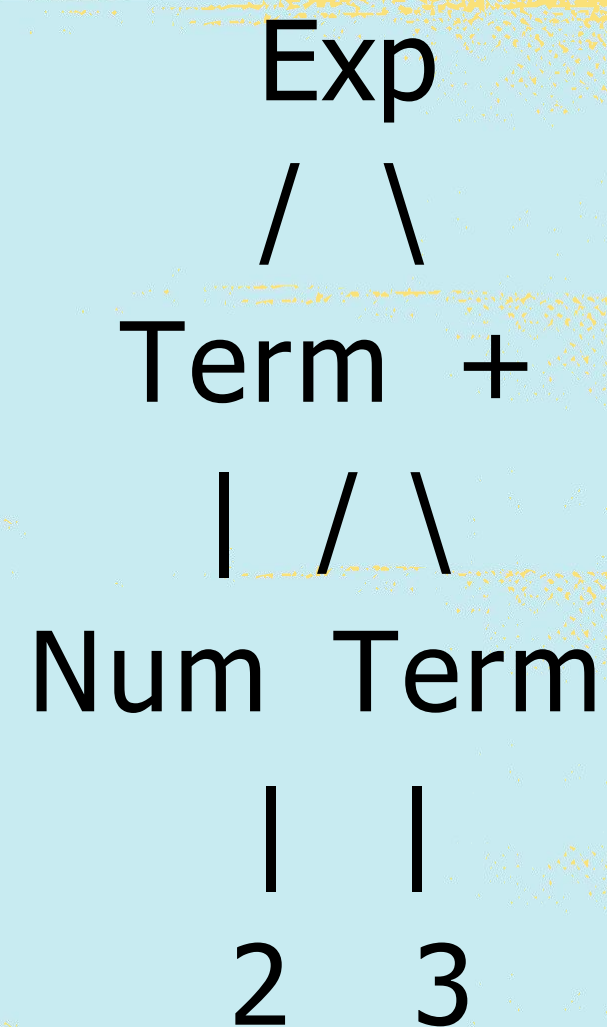
Term $\rightarrow$ Num

Donde "Exp" representa una expresión, "Term" representa un término, y "Num" representa un número.

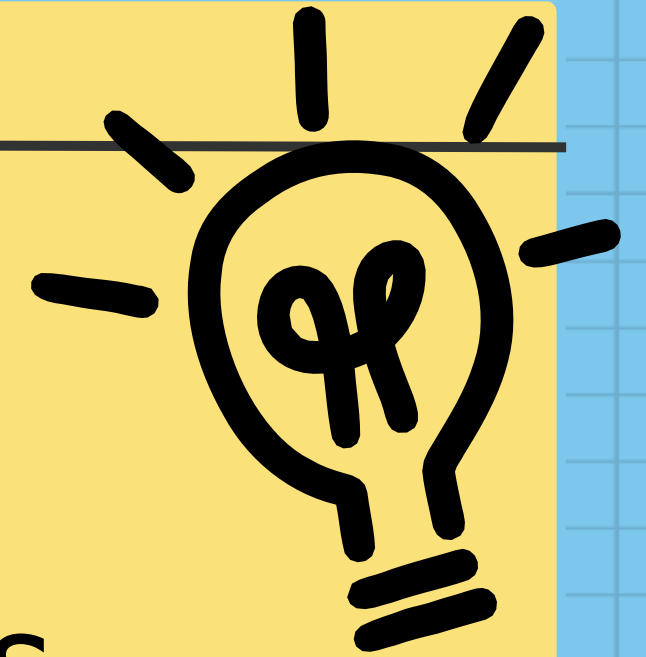
ÁRBOL SINTÁCTICO



Ahora, vamos a construir un árbol sintáctico para la expresión "2 + 3":

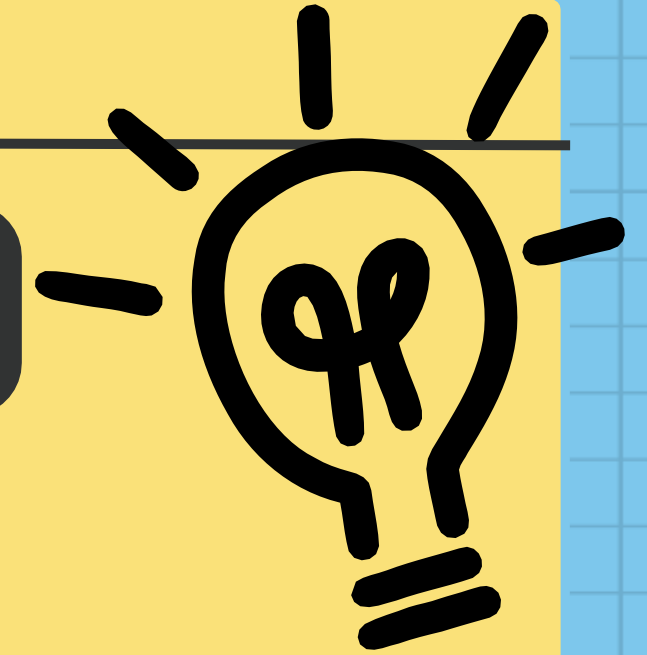


ACCIONES SEMÁNTICAS



Luego, aplicamos acciones semánticas para traducir este árbol sintáctico a inglés. Supongamos que tenemos acciones semánticas asociadas a cada regla de producción que simplemente concatenan las traducciones de sus hijos.

ACCIONES SEMÁNTICAS ASOCIADAS:



Para la regla $\text{Exp} \rightarrow \text{Exp} '+' \text{Term}$:

- $\text{Exp.english} = \text{Exp1.english} + \text{' plus ' + Term.english}$

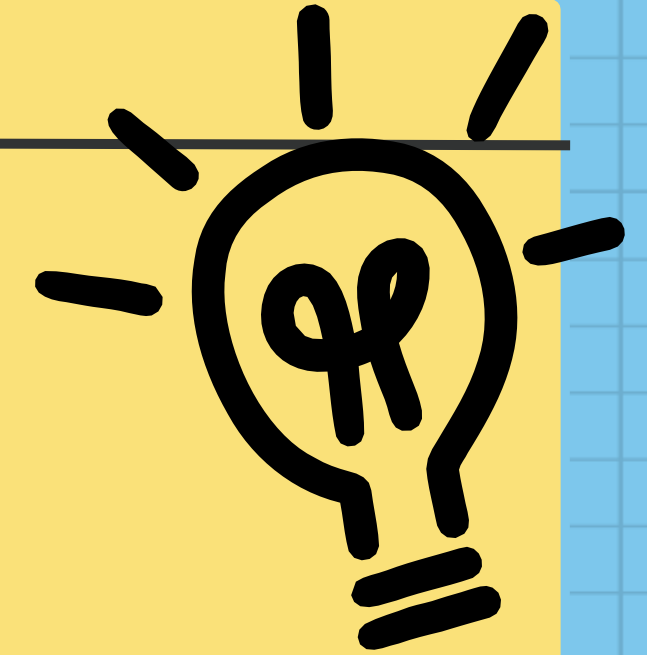
Para la regla $\text{Exp} \rightarrow \text{Term}$:

- $\text{Exp.english} = \text{Term.english}$

Para la regla $\text{Term} \rightarrow \text{Num}$:

- $\text{Term.english} = \text{Num.value}$

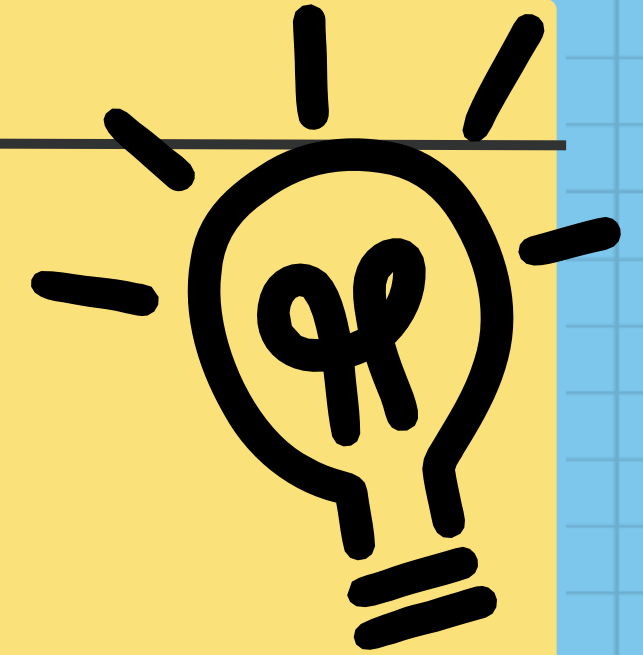
PASOS DE APLICACIÓN



- **Term -> Num (Num = 2):**

Se aplica la regla Term -> Num, y como Num es 2, se calcula el atributo semántico:
Term.english = 2

PASOS DE APLICACIÓN



- **Exp -> Term:**

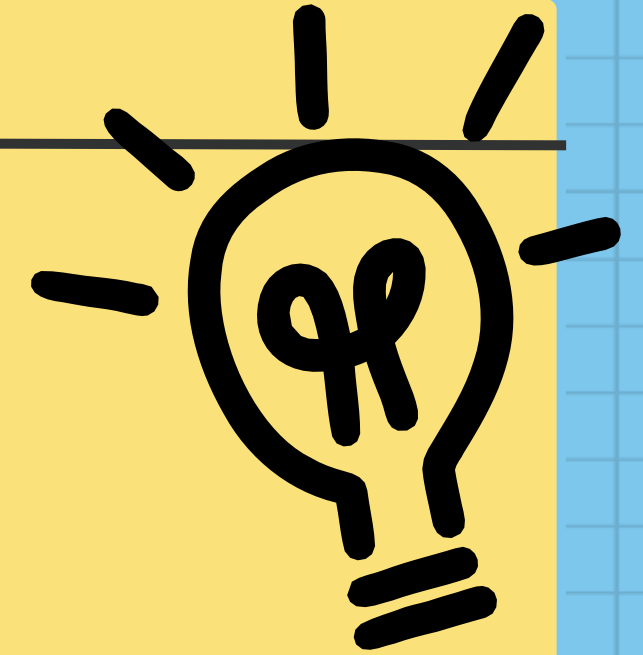
Se aplica la regla $\text{Exp} \rightarrow \text{Term}$, asignando el valor de Term.english a Exp.english :

$\text{Exp.english} = \text{Term.english}$

En este punto, el atributo semántico de la expresión es "2".



PASOS DE APLICACIÓN



- **Exp $\rightarrow$ Exp '+' Term:**

Se aplica la regla Exp $\rightarrow$ Exp '+' Term, concatenando los atributos semánticos de Exp1 y Term, y agregando la operación '+':

**Exp.english = Exp1.english + ' plus ' +
Term.english**

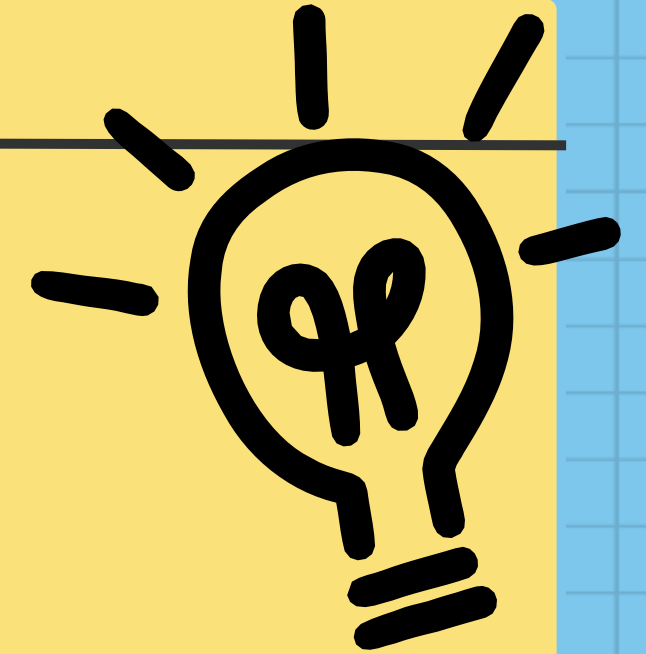
Se concatena todo para obtener la traducción en inglés:

Exp.english = '2' + ' plus ' + '3'

Exp.english = '2 plus 3'

Por lo tanto, la traducción al inglés de la expresión en español "2 + 3" utilizando las acciones semánticas especificadas sería "2 plus 3".

EJEMPLO 2



```
expr -> expr + term {  
    if (expr.tipo == term.tipo) {  
        expr.tipo = term.tipo;  
        expr.valor = expr.valor + term.valor;  
    } else {  
        reportarError("Tipos incompatibles en la suma");  
    }  
}
```


22

```
...  
...  
expr -> expr + term {  
    if (term.tipo == tipo) {  
        expr.valor = expr.valor + term.valor;  
    } else {  
        reportarError("Tipos incompatibles en la suma");  
    }  
}
```

expr representa una expresión.

22

22

...

...

```
expr -> expr + term {  
    if (expr.tipo == term.tipo;  
        expr.valor = expr.valor + term.valor;  
    } else {  
        reportarError("Tipos incompatibles en la suma");  
    }  
}
```

term representa otro término en la expresión.

...

...

```
expr -> expr + term {  
  if (expr.tipo == term.tipo,  
      expr.tipo == term.tipo,  
      expr.valor = expr.valor + term.valor;  
  } else {  
    reportarError("Tipos incompatibles en la suma");  
  }  
}
```

tipo que indica su tipo (por ejemplo, entero o flotante).

...

...

```
expr -> expr + term {  
    if (expr.tipo == term.tipo) {  
        expr.tipo = term.tipo;  
        expr.valor = expr.valor + term.valor;  
    }  
}
```

Si los tipos son compatibles, actualizamos el tipo de `expr` y realizamos la operación de suma de `expr.valor` y `term.valor` (suponiendo que ambos tienen un atributo `valor` que contiene su valor numérico).

}

↗

20

...

...

```
expr -> expr + term {  
    if (expr.tipo == term.tipo) {  
        expr.tipo = term.tipo;  
        expr.valor = expr.valor + term.valor;  
    } else {  
        reportarError("Tipos incompatibles en la suma");  
    }  
}
```

↗

20

...

```
expr -> expr + term {  
    if (expr.tipo == term.tipo) {  
        expr.tipo = term.tipo;  
        expr.valor = expr.valor + term.valor;  
    } else {  
        reportarError("Tipos incompatibles en la suma");  
    }  
}
```

Si los tipos no son compatibles, generamos un mensaje de error utilizando `reportarError` para indicar que se ha encontrado un error semántico.

...